Tratamineto de Video para Datos

March 24, 2026

Angel Manuel Gonzalez Lopez

1 Lectura y ajustar corte

[ ]:
```
video_path = 'video01.mp4'

cut_minx = 65
cut_maxx = 405
```

[ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow

# Cargar el video

cap = cv2.VideoCapture(video_path)

# Verificar si el video abrió correctamente
if not cap.isOpened():
    print("Error al abrir el video. Revisa el nombre del archivo.")
else:
   print("Cargado exitosaente")
```

```
Cargado exitosaente
```

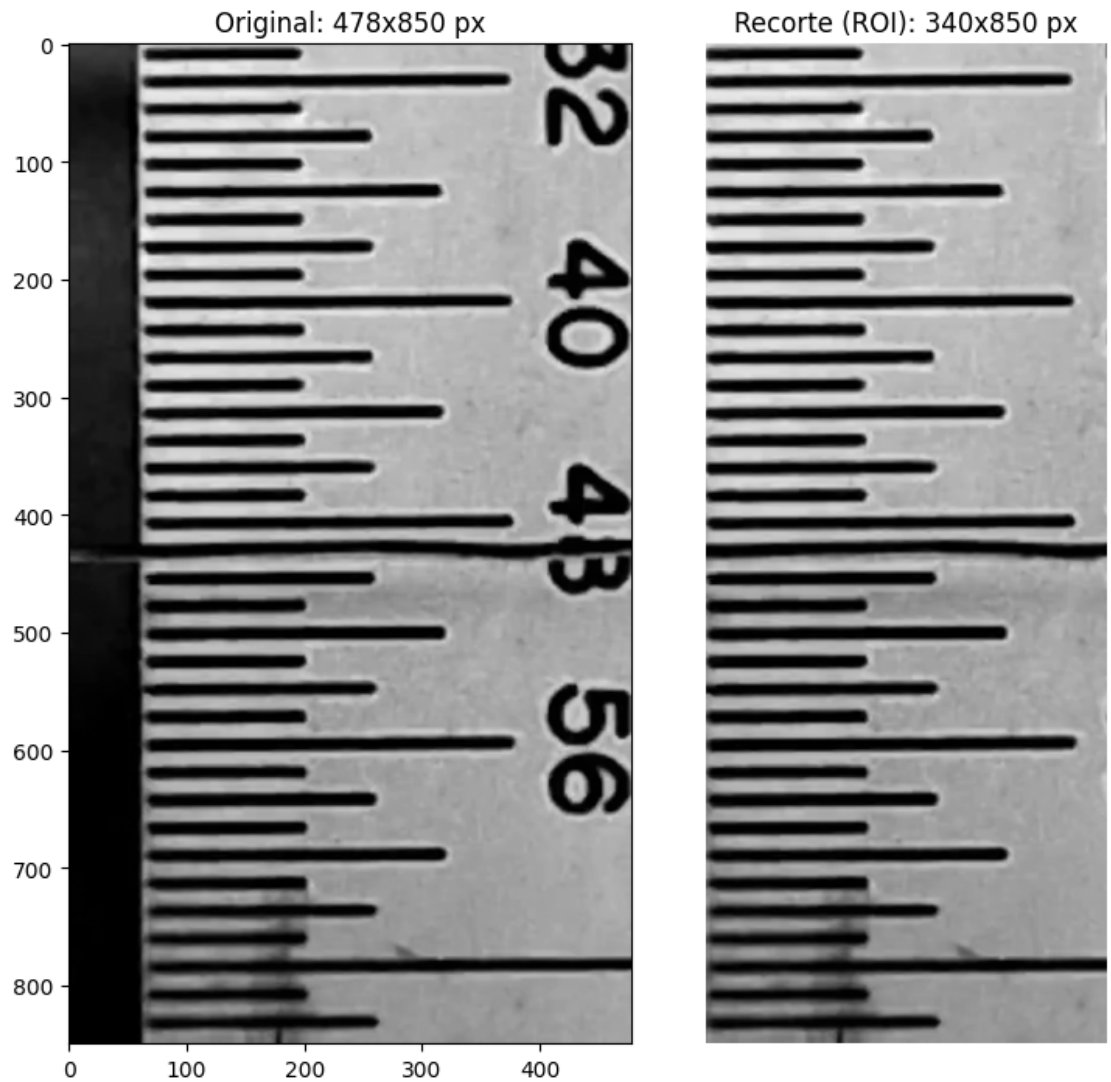
[ ]:
```
# Obtener el número total de cuadros
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# Obtener la tasa de cuadros por segundo (FPS)
fps = cap.get(cv2.CAP_PROP_FPS)

# Calcular duración en segundos
duracion = total_frames / fps

print(f"Total de cuadros: {total_frames}")
print(f"FPS: {fps}")
```

```
print(f"Duración: {duracion:.2f} segundos")
```

```
Total de cuadros: 12342
FPS: 29.996526086969673
Duración: 411.45 segundos
```

[ ]:
```
 # Leer solo el primer frame para configurar la "Zona de Interés" (ROI)
ret, frame = cap.read()

# Convertir a gris para facilitar la detección
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Mostramos el primer frame para que identifiques las coordenadas
print("Primer frame del video:")
print(f"dimensiones: { gray.shape[0] } x { gray.shape[1]} px ")

gray_cut = gray[:, cut_minx:cut_maxx]
print("aplicando corte")
print(f"dimensiones: { gray_cut.shape[0] } x { gray_cut.shape[1]} px ")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 7))
# Visualización del Frame Original
ax1.imshow(gray, cmap='gray')
ax1.set_title(f"Original: {gray.shape[1]}x{gray.shape[0]} px")
# ax1.axis('off') # Opcional: quita los ejes para mejor estética

# Visualización del Frame Recortado
ax2.imshow(gray_cut, cmap='gray')
ax2.set_title(f"Recorte (ROI): {gray_cut.shape[1]}x{gray_cut.shape[0]} px")
ax2.axis('off')

plt.tight_layout() # Ajusta automáticamente el espaciado entre gráficas
plt.show()
```

```
Primer frame del video:
dimensiones: 850 x 478 px
aplicando corte
dimensiones: 850 x 340 px
```



2 Encontrar Lineas

2.1 Ajustes previos

```
[ ]: def visualizar_detecciones(imagen_base, posiciones_y, col1 = (255, 0, 0) , col2␣
     ↪= (-1, 0, 0) ):
         """
         Dibuja líneas horizontales sobre la imagen para verificar la detección.
         """

         if col2 == (-1, 0, 0):
             col2 =col1
```

```
    # Si la imagen es en escala de grises, la convertimos a BGR para poder␣
↪dibujar en COLOR (Rojo)
    if len(imagen_base.shape) == 2:
        output = cv2.cvtColor(imagen_base, cv2.COLOR_GRAY2BGR)
    else:
        output = imagen_base.copy()

    # El ancho de la imagen para que la línea la cruce toda
    ancho = output.shape[1]

    for y in posiciones_y:
        # Dibujamos una línea roja (0, 0, 255) con grosor 2
        # cv2.line(imagen, (x_inicio, y_inicio), (x_fin, y_fin), color, grosor)
        cv2.line(output, (0, int(y)), (ancho, int(y)), col1, 2)

        # Opcional: Escribir el valor de Y al lado de la línea
        cv2.putText(output, str(int(y)), (10, int(y) - 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4,col2, 1)


    return output
```

[ ]:
```
def agrupar_lineas(posiciones_y, tolerancia=5):
    if not posiciones_y:
        return []

    #  Ordenar las posiciones de arriba hacia abajo
    pos_ordenadas = sorted(posiciones_y)

    lineas_reales = []
    if len(pos_ordenadas) > 0:
        grupo_actual = [pos_ordenadas[0]]

        for i in range(1, len(pos_ordenadas)):
            # Si la diferencia con la anterior es menor a la tolerancia (en␣
↪píxeles)
            if pos_ordenadas[i] - pos_ordenadas[i-1] <= tolerancia:
                grupo_actual.append(pos_ordenadas[i])
            else:
                # Calculamos el promedio del grupo anterior y lo guardamos
                lineas_reales.append(int(np.mean(grupo_actual)))
                grupo_actual = [pos_ordenadas[i]]

        # No olvidar el último grupo
        lineas_reales.append(int(np.mean(grupo_actual)))

    return lineas_reales
```

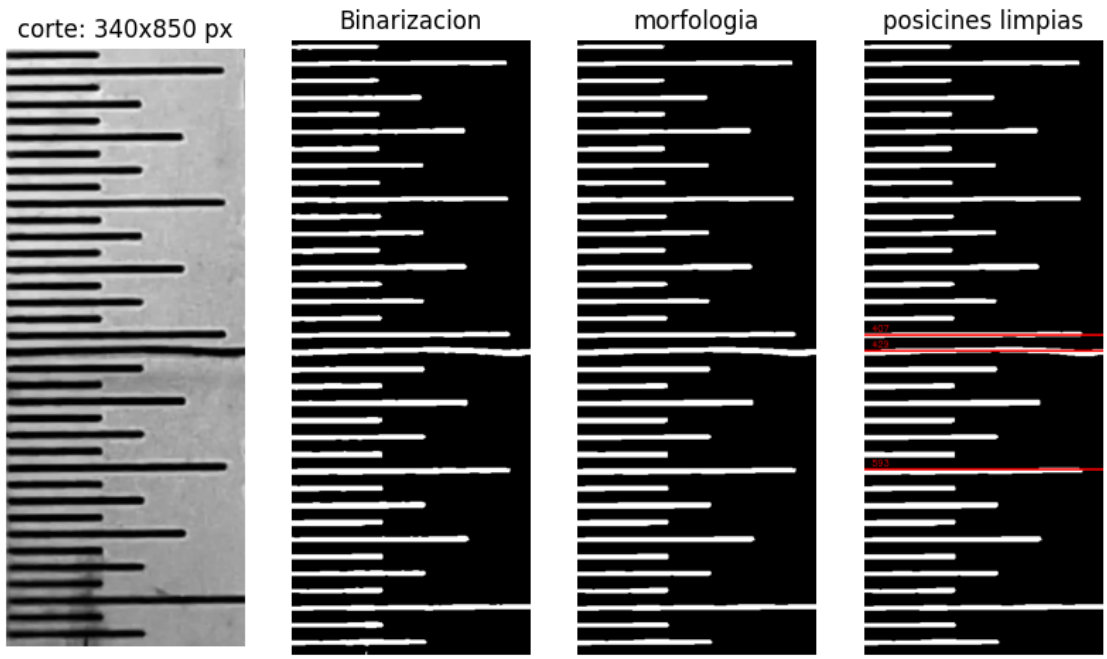
2.2 Pricesamiento completo

[ ]:
```
def procesar_medicion(frame, longitud=60,  tolerancia = 10, ver = False):


    # 0. Recorte
    roi = frame[:, cut_minx:cut_maxx]

    ancho = roi.shape[1]
    longitud  =  ancho* longitud// 100


    # 1. Umbralización (Binarización) para resaltar líneas negras
    # Usamos cv2.THRESH_BINARY_INV porque las marcas son oscuras sobre fondo␣
↪claro
    _, thresh = cv2.threshold(roi, 20, 255, cv2.THRESH_BINARY_INV ) #_INV
    thresh = thresh[:, 10:]
    # cv2_imshow(roi)

    # 2. Definir un elemento estructurante horizontal (un "peine")
    # Esto buscará estructuras que tengan al menos 30px de ancho y 1px de alto
    kernel = np.ones((1, 25), np.uint8)

    # 3. Aplicar "Opening" (Erosión seguida de Dilatación)
    # Esto borra puntos pequeños, números y ruido, dejando solo líneas
    morfologia = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)


    # 3. Detectar líneas horizontales (el palito y las marcas)
    lines = cv2.HoughLinesP(morfologia, 1, np.pi/180, threshold=50,␣
↪minLineLength=longitud, maxLineGap=5)

    posiciones_y = []
    if lines is not None:
        for line in lines:
            x1, y1, x2, y2 = line[0]
            # Si la línea es horizontal (y1 aprox igual a y2)
            if abs(y1 - y2) < 3:
                posiciones_y.append(y1)


    ## agruparlas
    posiciones_limpias = agrupar_lineas(posiciones_y, tolerancia=tolerancia)

    if ver:

      fig, ax = plt.subplots(1, 4, figsize=(10, 6))
```

```
        # Visualización del Frame Recortado
        ax[0].imshow(roi, cmap='gray')
        ax[0].set_title(f"corte: {roi.shape[1]}x{roi.shape[0]} px")
        ax[0].axis('off') # Opcional: quita los ejes para mejor estética

        #
        ax[1].imshow(thresh, cmap='gray')
        ax[1].set_title(f"Binarizacion")
        ax[1].axis('off')

        #
        ax[2].imshow(morfologia, cmap='gray')
        ax[2].set_title(f"morfologia")
        ax[2].axis('off')

        # #
        # imagen_verificacion = visualizar_detecciones(morfologia, posiciones_y)
        # ax[3].imshow(imagen_verificacion, cmap='gray')
        # ax[3].set_title(f"líneas detectadas")
        # ax[3].axis('off')

        #
        imagen_verificacion = visualizar_detecciones(morfologia,␣
 ↪posiciones_limpias)
        ax[3].imshow(imagen_verificacion, cmap='gray')
        ax[3].set_title(f"posicines limpias")
        ax[3].axis('off')


    return posiciones_limpias, morfologia
```

[ ]:
```
# Prueba con el primer frame
posiciones, imagen_binaria = procesar_medicion(gray, longitud = 80 , tolerancia␣
 ↪= 10, ver = True)

print(f"Se detectaron { len(posiciones) } lineas")
print(f"Líneas detectadas en posiciones Y: {posiciones}")
```

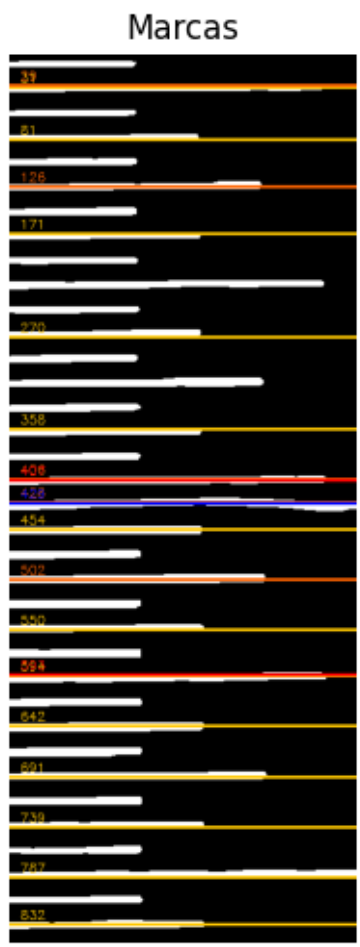
```
Se detectaron 3 lineas
Líneas detectadas en posiciones Y: [407, 429, 593]
```



3 Ver Lineas

```
[ ]: def verMarcas( img):
        posiciones_marcas_all, imagen_binaria = procesar_medicion(img, longitud =␣
     ↪40 , tolerancia = 10, ver = False)
        posiciones_marcas_med, imagen_binaria = procesar_medicion(img, longitud =␣
     ↪60 , tolerancia = 10, ver = False)
        posiciones_marcas_lar, imagen_binaria = procesar_medicion(img, longitud =␣
     ↪80 , tolerancia = 10, ver = False)
        posicion_vara, _ = procesar_medicion(img, longitud = 90 , tolerancia = 10,␣
     ↪ver = False)


        visualizar = visualizar_detecciones(imagen_binaria, posiciones_marcas_all,␣
     ↪col1=(255, 200, 0))
        visualizar = visualizar_detecciones(visualizar, posiciones_marcas_med,␣
     ↪col1=(255, 100, 0))
        visualizar = visualizar_detecciones(visualizar, posiciones_marcas_lar,␣
     ↪col1=(255, 0, 0))
        visualizar = visualizar_detecciones(visualizar, posicion_vara, col1=(0,0,␣
     ↪255))

        fig, ax = plt.subplots(1, 1, figsize=(3, 6))
        ax.imshow(visualizar, cmap='gray')
```

```
    ax.set_title(f"Marcas")
    ax.axis('off')

verMarcas(gray)
```



[ ]:
```
import imageio
import cv2
import numpy as np

frames_gif = []


video_path = 'video01.mp4'
cap = cv2.VideoCapture(video_path)
```

```python
# --- CONFIGURACIÓN ---
cap = cv2.VideoCapture('video01.mp4')
posiciones_y_crudo = []
tiempos = []

frame_id = 0
PASO_FRAMES = 50 # Procesa 1 de cada 10 frames del video original

frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print(f"Hay un totoal de {frames} frmes, se tomara 1 cada {skip_frames} ")
print(f"comenzando a procesar un totoal de {frames//PASO_FRAMES} imagenes")


# Configuración de FPS para el GIF final
fps_video = cap.get(cv2.CAP_PROP_FPS)
if fps_video <= 0:
    fps_video = 30 # Valor por defecto si falla la detección
    print("Aviso: No se detectaron FPS, usando 30 por defecto.")



print("Generando frames para el GIF animado...")

frame_id = 0 # Contador global de frames leídos
frames_procesados_contador = 0 # Contador de frames guardados para el GIF

# --- USAMOS EL BUCLE WHILE COMO EN LA GRÁFICA ---
while cap.isOpened():
    ret, frame = cap.read()

    # Si no hay más frames, salimos del bucle
    if not ret:
        break

    # Solo procesamos si corresponde según el PASO_FRAMES
    if frame_id % PASO_FRAMES == 0:

        ### PROCESAR ---------------------
        # Asegúrate de tener definidas cut_minx y cut_maxx
        try:
            roi_color = frame[:, cut_minx:cut_maxx].copy()
            gray_f = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        except NameError:
            # Si no definiste los cortes, procesa el frame completo
            roi_color = frame.copy()
            gray_f = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
            print("Aviso: cut_minx o cut_maxx no definidos, usando frame␣
↪completo.")
            # Definir cortes por defecto para evitar error en bucle
            cut_minx, cut_maxx = 0, frame.shape[1]


        ### DETECTAR ---------------------


        # Lógica de Detección por Jerarquías (como en tu verMarcas)
        # Usamos gray_f para procesar_medicion
        p_all, _ = procesar_medicion(gray_f, longitud=40)
        p_med, _ = procesar_medicion(gray_f, longitud=60)
        p_lar, _ = procesar_medicion(gray_f, longitud=80)
        p_vara, _ = procesar_medicion(gray_f, longitud=95)

        # Dibujar líneas de colores sobre el ROI a color
        # Asegúrate de que visualizar_detecciones esté definida correctamente
        dibujo = visualizar_detecciones(roi_color, p_all, col1=(255, 200, 0)) #␣
↪Amarillo (all)
        dibujo = visualizar_detecciones(dibujo, p_med, col1=(255, 100, 0))  #␣
↪Naranja (med)
        dibujo = visualizar_detecciones(dibujo, p_lar, col1=(255, 0, 0))    #␣
↪Rojo (lar)
        dibujo = visualizar_detecciones(dibujo, p_vara, col1=(0, 0, 255))   #␣
↪Azul (vara)


        ### TEXTO FRAME ---------------------


        # Definimos el texto, la fuente, escala, color (BGR) y grosor
        texto_frame = f"Frame: {frame_id}"
        fuente = cv2.FONT_HERSHEY_SIMPLEX
        posicion_texto = (10, 30) # Esquina superior izquierda (x, y)
        cv2.putText(dibujo, texto_frame, posicion_texto, fuente, 1, (255, 255,␣
↪255), 2, cv2.LINE_AA)

        # Convertir de BGR (OpenCV) a RGB (ImageIO)
        dibujo_rgb = cv2.cvtColor(dibujo, cv2.COLOR_BGR2RGB)
        frames_gif.append(dibujo_rgb)

        frames_procesados_contador += 1
        # Imprimir progreso cada 20 frames procesados
        if frames_procesados_contador % 50 == 0:
```

```
            print(f"Frames procesados y guardados:␣
↪{frames_procesados_contador}")



    frame_id += 1 # Incrementar el contador global
    # Opcional: Límite para que el GIF no tarde eternamente en la prueba
    # if frame_id > 2000: break

cap.release()
```

```
Hay un totoal de 12342 frmes, se tomara 1 cada 30
comenzando a procesar un totoal de 246 imagenes
Generando frames para el GIF animado…
Frames procesados y guardados: 30
Frames procesados y guardados: 60
Frames procesados y guardados: 90
Frames procesados y guardados: 120
Frames procesados y guardados: 150
Frames procesados y guardados: 180
Frames procesados y guardados: 210
Frames procesados y guardados: 240
¡GIF guardado con éxito como 'resultado_rastreo_marcas.gif'!
```

[ ]:
```
# Guardar el GIF animado
if len(frames_gif) > 0:
    nombre_archivo_gif = 'resultado_rastreo_marcas.gif'
    # Ajustamos los FPS del GIF dividiendo por el paso de frames para mantener␣
↪velocidad real
    # fps_salida = fps_video / PASO_FRAMES
    fps_salida = fps_video / PASO_FRAMES * 6 # pasa de 400 s a  1min aporxxx #␣
↪<---------------------
    imageio.mimsave(nombre_archivo_gif, frames_gif, fps=fps_salida, loop=0)
    print(f"¡GIF guardado con éxito como '{nombre_archivo_gif}'!")
else:
    print("Error: No se capturó ningún frame. Revisa la ruta del video o los␣
↪parámetros de detección.")
```

```
¡GIF guardado con éxito como 'resultado_rastreo_marcas.gif'!
```

4 Primera aproximacion

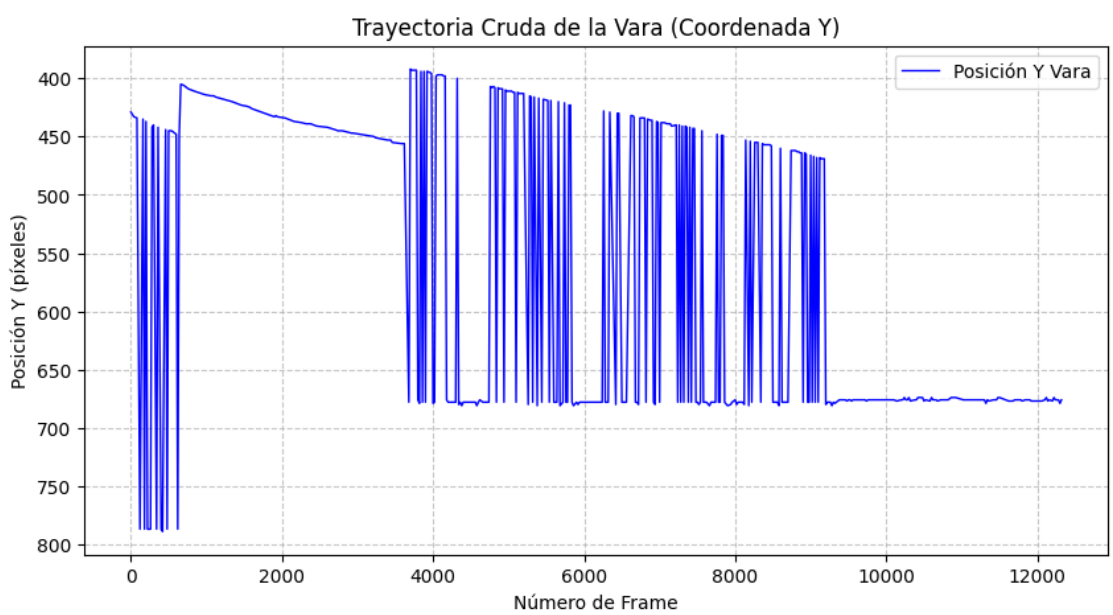
[ ]:
```
import matplotlib.pyplot as plt
import pandas as pd

# --- CONFIGURACIÓN ---
```

```python
cap = cv2.VideoCapture('video01.mp4')
posiciones_y_crudo = []
tiempos = []

frame_id = 0
skip_frames = 20 # Procesar 1 de cada 20 para ir más rápido

frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print(f"Hay un totoal de {frames} frmes, se tomara 1 cada {skip_frames} ")
print(f"comenzando a procesar un totoal de {frames//skip_frames} imagenes")

cnt = 0
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    if frame_id % skip_frames == 0:
        # Convertir a gris para tu función
        gray_f = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # funcion ya implemenada
        posicion_vara, _ = procesar_medicion(gray_f, longitud=95,␣
 ↪tolerancia=10, ver=False)

        # Si detectó algo, guardamos la primera posición encontrada
        if len(posicion_vara) > 0:
            posiciones_y_crudo.append(posicion_vara[0])
            tiempos.append(frame_id)


        cnt += 1
        if cnt % 50 == 0 :
            print(f"se han procesaron {cnt} imagenes...")

    frame_id += 1

    # Opcional: poner un límite si el video es muy largo para la primera prueba
    # if frame_id > 1000: break

cap.release()
```

```
Hay un totoal de 12342 frmes, se tomara 1 cada 20
comenzando a procesar un totoal de 617 imagenes
se han procesaron 50 imagenes…
se han procesaron 100 imagenes…
se han procesaron 150 imagenes…
```

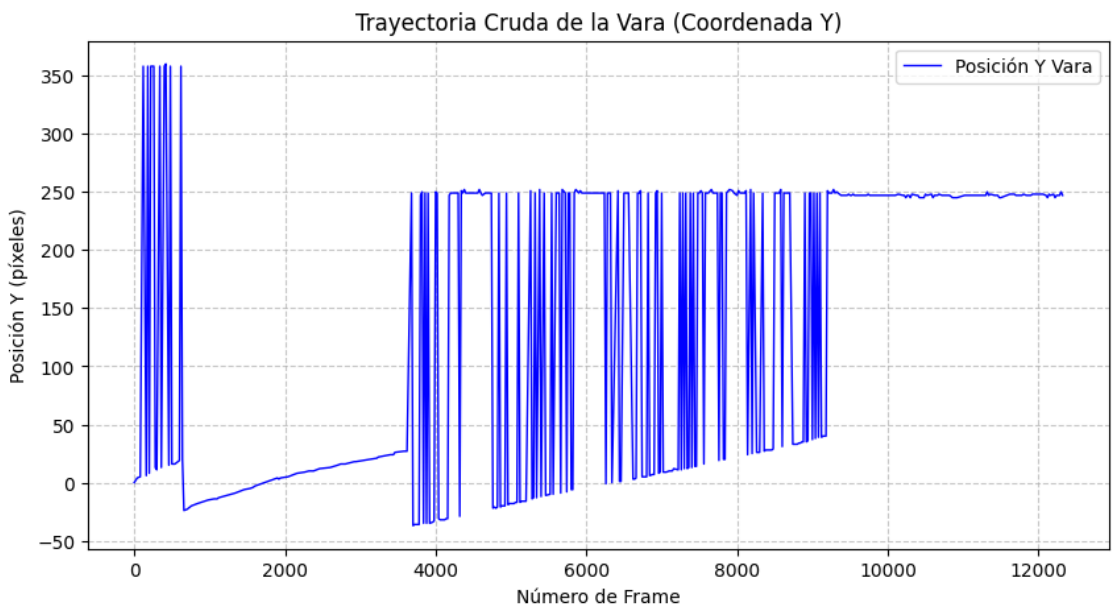
```
se han procesaron 200 imagenes…
se han procesaron 250 imagenes…
se han procesaron 300 imagenes…
se han procesaron 350 imagenes…
se han procesaron 400 imagenes…
se han procesaron 450 imagenes…
se han procesaron 500 imagenes…
se han procesaron 550 imagenes…
se han procesaron 600 imagenes…
```

[ ]:
```
# --- GENERAR GRAFICA ---
plt.figure(figsize=(10, 5))
plt.plot(tiempos, posiciones_y_crudo, color='blue', linewidth=1,␣
 ↪label='Posición Y Vara')

# Invertimos el eje Y porque en las imágenes el 0 está arriba
plt.gca().invert_yaxis()

plt.title('Trayectoria Cruda de la Vara (Coordenada Y)')
plt.xlabel('Número de Frame')
plt.ylabel('Posición Y (píxeles)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()
plt.show()

print(f"Gráfica generada con {len(posiciones_y_crudo)} puntos.")
```



```
Gráfica generada con 390 puntos.
```

[271]:
```
data = np.array(posiciones_y_crudo, dtype=float)
data = data - data[0]

# --- GENERAR GRAFICA ---
plt.figure(figsize=(10, 5))
plt.plot(tiempos, data, color='blue', linewidth=1, label='Posición Y Vara')


plt.title('Trayectoria Cruda de la Vara (Coordenada Y)')
plt.xlabel('Número de Frame')
plt.ylabel('Posición Y (píxeles)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()
plt.show()

print(f"Gráfica generada con {len(data)} puntos.")
```



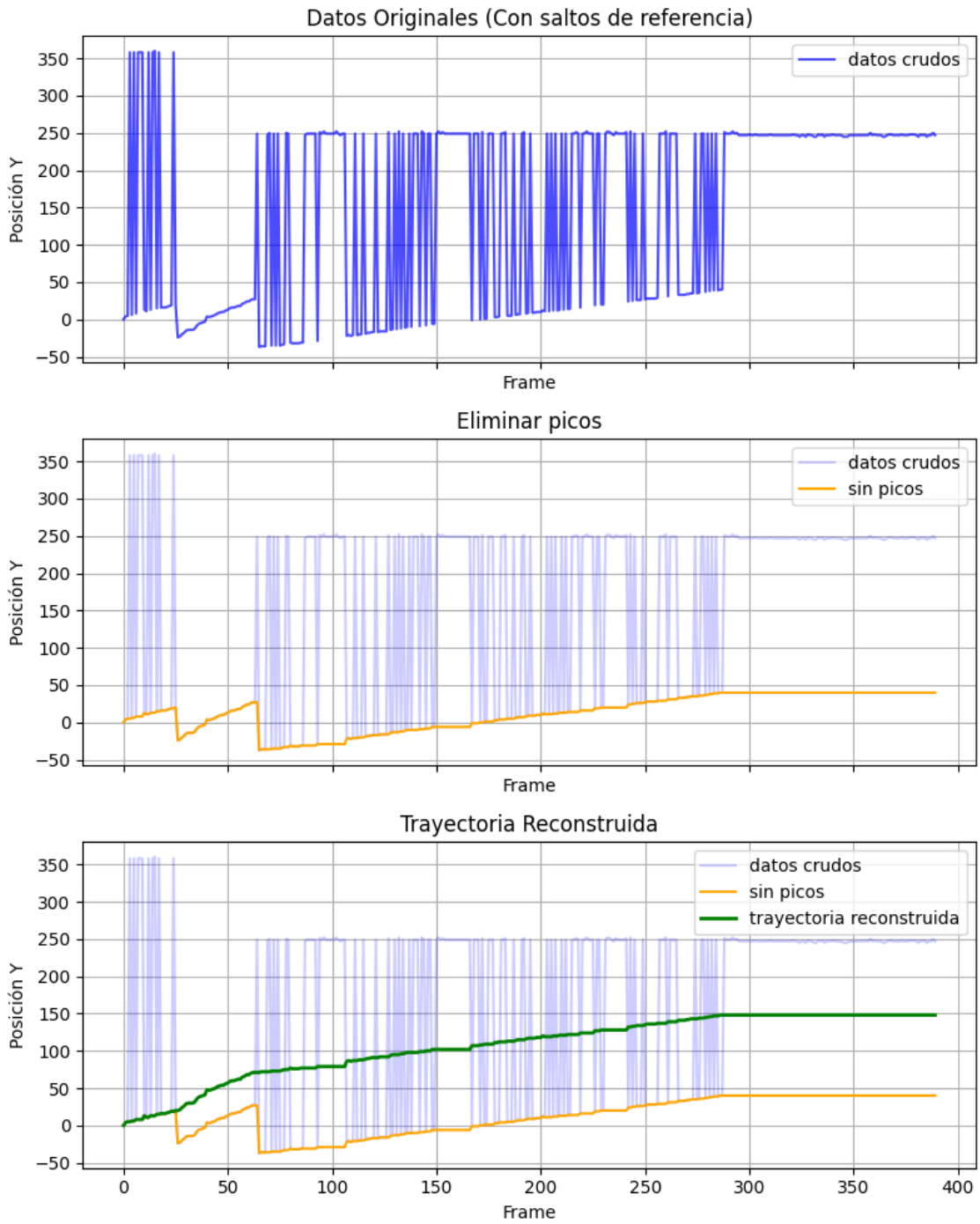

```
Gráfica generada con 390 puntos.
```

[342]:
```
import numpy as np
import matplotlib.pyplot as plt

def limpiar_y_trasladar(datos_crudos, umbral_ruido=600, umbral_salto=20,␣
↪ver=False):
```

```
    # 1. Filtro inicial: Quitar picos absolutos (outliers de detección)
    # Reemplazamos por el valor anterior para no perder la secuencia
    data = np.array(datos_crudos, dtype=float)
    # data = data - data[0]
    for i in range(len(data)):
        if data[i] > umbral_ruido:
            data[i] = data[i-1] if i > 0 else data[i]

    # 2. Corrección por traslación (Reconectar la trayectoria)
    data_corregida = np.copy(data)
    offset_acumulado = 0

    for i in range(1, len(data_corregida)):
        # Aplicamos el offset acumulado hasta ahora
        data_corregida[i] -= offset_acumulado

        # Calculamos la diferencia con el punto anterior ya corregido
        salto = data_corregida[i] - data_corregida[i-1]

        # Si el salto es mayor al umbral, es un movimiento de la regla/cámara
        if abs(salto) > umbral_salto:
            offset_acumulado += salto
            data_corregida[i] -= salto # Corregimos el punto actual␣
↪inmediatamente


    if ver:

        fig, axs = plt.subplots(3,1, figsize=(8, 10), sharex=True)
        axs_flat = axs.flatten()

        # 1
        axs_flat[0].set_title("Datos Originales (Con saltos de referencia)")
        axs_flat[0].plot(datos_crudos, label='datos crudos', color='blue',␣
↪alpha=0.7)

        #
        axs_flat[1].set_title("Eliminar picos")
        axs_flat[1].plot(datos_crudos, label='datos crudos', color='blue',␣
↪alpha=0.2)
        axs_flat[1].plot(data, label='sin picos', color='orange')


        #
        axs_flat[2].set_title("Trayectoria Reconstruida")
```

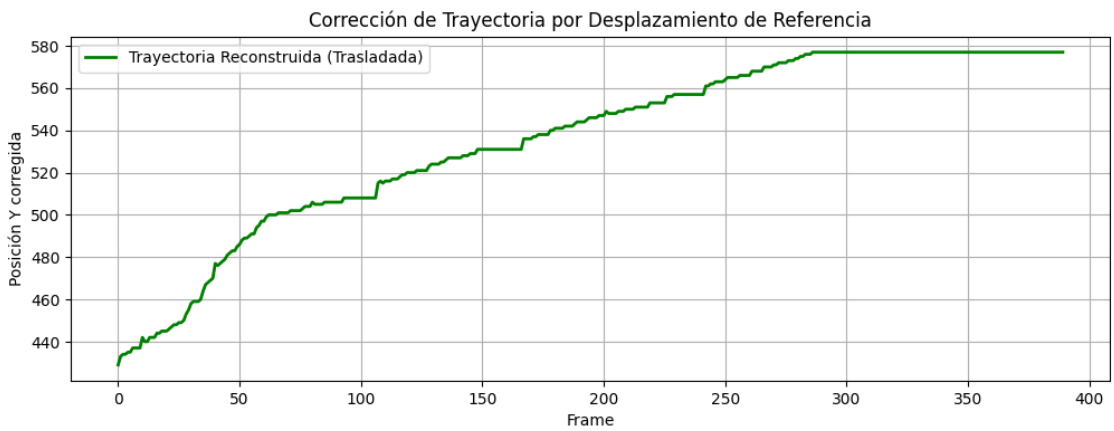
```
        axs_flat[2].plot(datos_crudos, label='datos crudos', color='blue',␣
↪alpha=0.2)
        axs_flat[2].plot(data, label='sin picos', color='orange')
        axs_flat[2].plot(data_corregida, label='trayectoria reconstruida',␣
↪color='green', linewidth=2)


        for i in range(3):
            axs_flat[i].set_ylabel("Posición Y")
            axs_flat[i].set_xlabel("Frame")
            axs_flat[i].grid(True)
            axs_flat[i].legend()

        # Ajustar el diseño para evitar que se solapen los títulos
        plt.tight_layout()
        plt.show()

    return data_corregida
```

[356]:
```
# Ejecución

lolol = np.array(posiciones_y_crudo, dtype=float)
lolol = lolol - lolol[0]
y_final = limpiar_y_trasladar(lolol, umbral_ruido=200, umbral_salto=20,␣
↪ver=True)
```



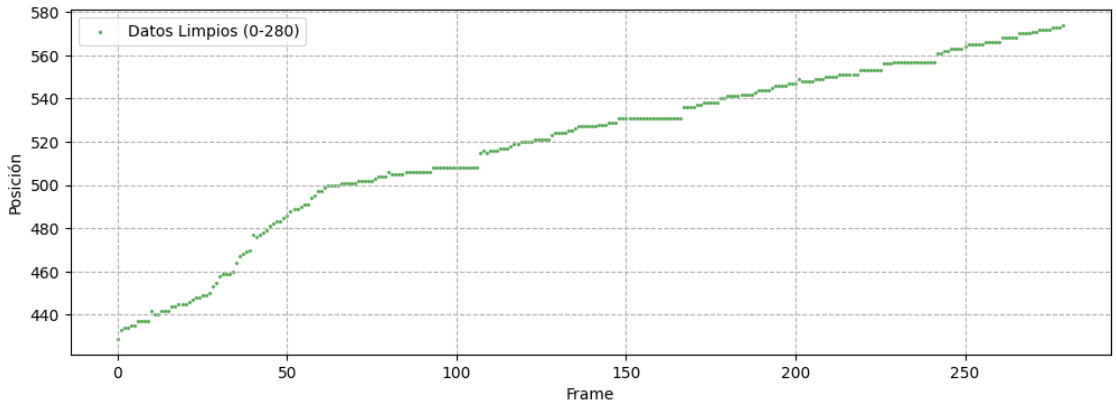
[349]:
```
# Visualización
plt.figure(figsize=(12, 4))

# plt.plot(data, label='Crudo (Movimiento de regla)', alpha=0.5, color='blue')
```

```
plt.plot(y_final, label='Trayectoria Reconstruida (Trasladada)', color='green',␣
↪linewidth=2)
# plt.gca().invert_yaxis()

plt.title("Corrección de Trayectoria por Desplazamiento de Referencia")
plt.ylabel("Posición Y corregida")
plt.xlabel("Frame")

plt.legend()
plt.grid(True)
plt.show()
```



[348]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# 1. Recortamos los datos hasta el frame 280
frame_limite = 280
x_data = np.arange(frame_limite)
y_data = np.array(y_final[:frame_limite])


# 5. Graficar
plt.figure(figsize=(12, 4))
plt.scatter(x_data, y_data, s=2, color='green', label='Datos Limpios (0-280)',␣
↪alpha=0.5)

plt.xlabel("Frame")
plt.ylabel("Posición")
plt.legend()
plt.grid(True, linestyle='--')
```

```
plt.show()
```


Datos Limpios (0-280)
580
560
540
520
500
480
460
440
Posición
0
50
100
150
200
250
Frame